



BTS SIO SLAM – Rapport de stage 1ère année

Développement d'un site web complet en autonomie

Mohamed Belmokhtar

Agence Web Conseil
19 mai au 20 juin 2025

Sommaire

1	Présentation de l'entreprise	3
1.1	Fiche d'identité	3
1.2	Organisation	3
2	Missions effectuées	4
2.1	Objectif principal	4
2.2	Tâches réalisées	4
2.3	Exemples de réalisations	7
2.4	Lien avec les compétences du BTS	7
A	Annexes	8
A.1	Extrait de code : Création de table SQL	
A.2	Extrait de code : Hachage du mot de passe avec sel	
A.3	Extrait de code : Attribution du JWT	

Introduction

Ce rapport présente l'expérience professionnelle que j'ai acquise lors d'un stage de six semaines effectuées au sein de l'agence Agence Web Conseil, spécialisée dans le développement de solutions web. Mon objectif principal était de participer à la création complète d'un site web, du développement frontend au backend, en intégrant notamment un système d'authentification sécurisé basé sur des tokens de session. Ce stage m'a permis de développer plusieurs compétences techniques essentielles, qui représentent un véritable atout dans mon parcours professionnel et qui m'aideront à concrétiser mon projet de devenir DevOps.

1 Présentation de l'entreprise

1.1 Fiche d'identité

- **Nom** : Agence Web Conseil
- **Domaine** : Développement web accompagnement digital–
- Localisation** : Neuilly-Sur-Marne, France
- **Effectif** : 5 à 10 personnes
- **Activité** : Création de sites vitrine, e-commerce, et formation.

1.2 Organisation

L'entreprise fonctionne en petite équipe selon une organisation inspirée des méthodes agiles, privilégiant la communication, la réactivité et l'adaptation continue aux besoins du projet. Cette approche permet d'assurer une meilleure collaboration entre les membres de l'équipe et une flexibilité dans la gestion des tâches.

Par exemple, des points réguliers (type stand-up meetings) facilitaient le suivi de l'avancement et la résolution rapide des blocages. L'agilité offre ainsi l'avantage de favoriser l'autonomie, la responsabilisation et la créativité, tout en permettant d'obtenir un produit plus proche des attentes des utilisateurs.

Mon tuteur était développeur full-stack senior. Compte tenu de mon niveau technique, j'ai été placé en autonomie sur deux projets que j'ai moi-même imaginés, ce qui m'a permis de mettre en pratique cette démarche agile tout en développant mon sens de l'organisation et de la prise d'initiative.

2 Missions effectuées

2.1 Objectif principal

Concevoir un site web complet (front-end et back-end) en lien avec mes aspirations professionnelles, tout en laissant libre cours à ma créativité et en évitant de brider mon esprit lors de sa conception.

2.2 Tâches réalisées

- **Phase de cadrage** : définition des fonctionnalités, wireframes UX, mini cahier des charges
- **Développement Front-End** :HTML5, CSS3, JavaScript pour validation des formulaires et interactivité.
- **Développement Back-End** :
 - Serveur Node.js/Express
 - Deux BDD SQLite : ourApp.app (Users) + ctf.db (challenges).
 - Encapsulation SQL via challengeService.js
 - Schéma SQL → Annexe A .1.
- **Sécurité et authentification** :
 - Hashage des mots de passe avec bcrypt + sel → Annexe A.2
 - Authentification avec JWT stockée en cookie http-Only / SameSite=Strict → Annexe A.3.
 - Expiration après 24h.
 - Flags jamais exposés côté API.
- **Tests et validation** :
 - Outils navigateur (console, onglet réseau)
 - POSTMAN pour tester les routes (login, register, verifyFlag).
 - Vérification SQL (unicité email/username)

2.3 Exemples de réalisations

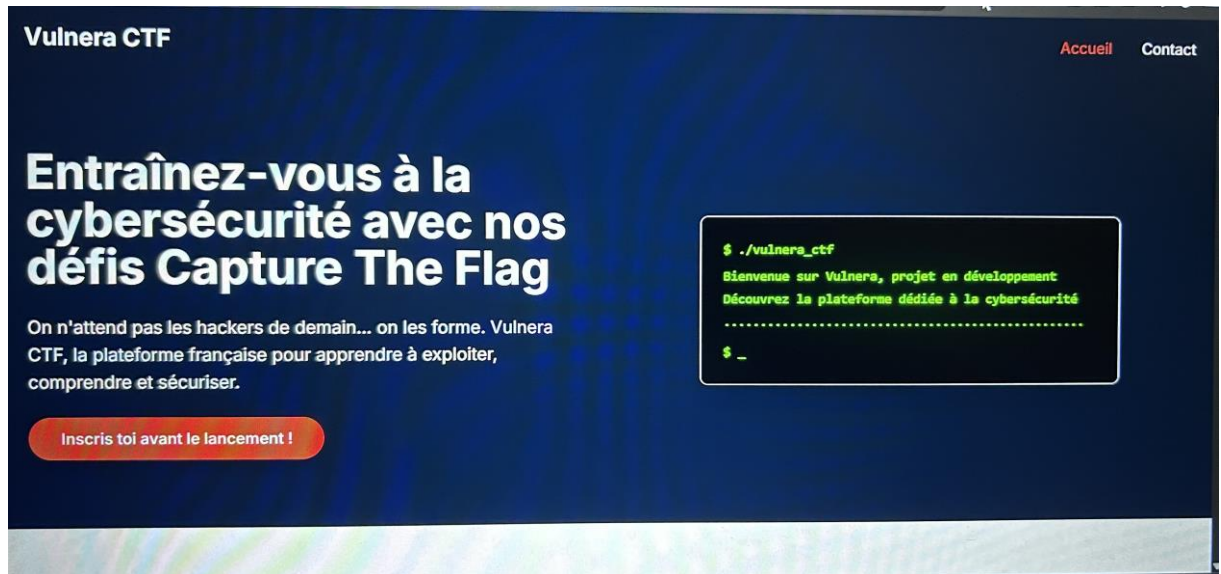


Figure 1 : Page d'accueil du site développé (version desktop).

2.4 Lien avec les compétences du BTS

Lors de ce stage, j'ai pu développer plusieurs compétences techniques directement en lien avec les attendus du BTS SIO, notamment dans l'option SLAM.

— Gestion des cookies et des sessions (JWT en cookie HTTP-Only) :

- **Authentification (Login/Middleware/logout)** : méthode expliquée (stateless JWT en cookie sécurisé) + solution illustrée (Annexes A.2 et A.3)
- **Schéma (DR6)** :
 - Schéma de données (tables users et challenges)
 - Schéma de sequence (cycle login → middleware → logout).
- **Exemple de requête optimisée** :
 - POST /api/challenges/:id/verify → projection minimale (SELECT id,title,...), pas de flag renvoyé
 - Optimisation : requête preparees, index implicites, WAL active. → ANNEXE A.5.
- **Division de code (DR7)** : séparation server.js (API/routes) et challengeService.js (accès SQL) = aintenabilité , sécurité, testabilité. → ANNEXE A.4.
- **Lien avec compétences BTS SIO SLAM (DR8)** :
 - Développement applicatif complet (front/back).
 - Gestion du patrimoine de données (SQL, optimisation, MERISE).
 - Sécurisation (JWT, bcrypt, bonnes pratiques).
 - Débogage et validation qualité.

Conclusion

Ce stage chez Agence Web Conseil a constitué bien plus qu'une simple application des connaissances acquises en formation : il a été une véritable immersion dans la réalité du métier de développeur. En participant à la conception complète d'un site web, du front-end jusqu'à la gestion sécurisée des sessions, j'ai pu mesurer l'importance de la rigueur, de l'autonomie et de la capacité d'adaptation dans un environnement professionnel agile.

Au-delà des compétences techniques consolidées — notamment en structuration de base de données, en gestion de sessions, ou en débogage —, cette expérience m'a permis de prendre confiance en ma capacité à mener un projet web complet et fonctionnel, tout en respectant les contraintes et les exigences réelles du terrain.

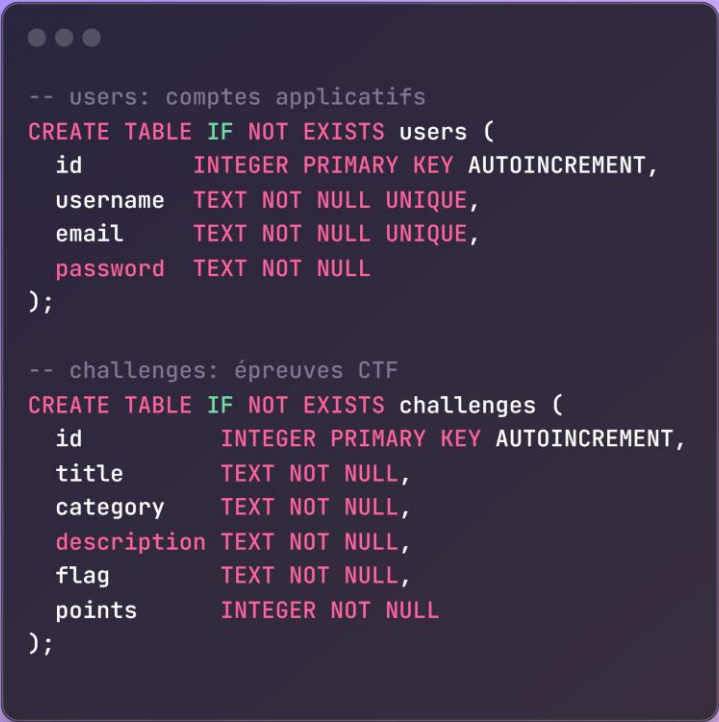
Ce stage a également été un révélateur : il a confirmé mon attrait pour des domaines spécifiques comme la sécurité des applications, l'architecture full-stack et l'amélioration de l'expérience utilisateur. Il a nourri mon envie d'aller plus loin, de sortir du cadre scolaire pour m'investir dans des projets concrets à impact.

Désormais, je souhaite poursuivre ce chemin par une alternance, pour continuer à apprendre aux côtés de professionnels, tout en construisant mon projet professionnel. Mon ambition à terme est de développer mes propres solutions numériques, en conjuguant innovation, maîtrise technique et utilité concrète.

En somme, ce stage a marqué un cap : il a transformé des savoirs théoriques en compétences pratiques, et une passion en projet d'avenir.

A Annexes

A.1 Schéma SQL (DDL) – utilisateurs & challenges



```
-- users: comptes applicatifs
CREATE TABLE IF NOT EXISTS users (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  username    TEXT NOT NULL UNIQUE,
  email       TEXT NOT NULL UNIQUE,
  password    TEXT NOT NULL
);

-- challenges: épreuves CTF
CREATE TABLE IF NOT EXISTS challenges (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  title       TEXT NOT NULL,
  category    TEXT NOT NULL,
  description TEXT NOT NULL,
  flag        TEXT NOT NULL,
  points      INTEGER NOT NULL
);
```

Commentaire :

- Unicité sur **username** et **email** (indices implicites SQLite) → recherche rapide au login et à l'inscription.
- Séparation **users** / **challenges** → sécurité (le flag reste côté serveur) maintenabilité.

A.2 Hashage mot de passe (bcrypt + sel)

```
-- users: comptes applicatifs
CREATE TABLE IF NOT EXISTS users (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  username    TEXT NOT NULL UNIQUE,
  email       TEXT NOT NULL UNIQUE,
  password    TEXT NOT NULL
);

-- challenges: épreuves CTF
CREATE TABLE IF NOT EXISTS challenges (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  title       TEXT NOT NULL,
  category    TEXT NOT NULL,
  description TEXT NOT NULL,
  flag        TEXT NOT NULL,
  points      INTEGER NOT NULL
);
```

Commentaire :

- Unicité sur **username** et **email** (indices implicites SQLite) → recherche rapide au login et à l'inscription.
- Séparation **users** / **challenges** → sécurité (le flag reste côté serveur) et maintenabilité.

A.3 JWT en cookie HTTP-Only (émission, middleware, logout)

```
// server.js - /api/auth/login (extrait)
const user = db.prepare("SELECT * FROM users WHERE email = ?").get(email);
if (!user || !bcrypt.compareSync(password, user.password)) {
  return res.status(400).json({ error: "Email ou mot de passe incorrect." });
}

const token = jwt.sign(
  { exp: Math.floor(Date.now() / 1000) + 60 * 60 * 24, userid: user.id, username:
  user.username },
  process.env.JWTSECRET
);

res.cookie("ourSimpleApp", token, {
  httpOnly: true,
  secure: process.env.NODE_ENV === 'production',
  sameSite: "strict",
  maxAge: 1000 * 60 * 60 * 24 // 24h
});
res.status(200).json({ message: "Connexion réussie" });

// Middleware global - vérification transparente du token
app.use((req, res, next) => {
  try {
    const token = req.cookies.ourSimpleApp;
    req.user = token ? jwt.verify(token, process.env.JWTSECRET) : null;
  } catch {
    req.user = null;
  }
  res.locals.user = req.user;
  next();
});

// /api/auth/me - route protégée
app.get('/api/auth/me', (req, res) => {
  if (!req.user) return res.status(401).json({ message: "Non authentifié" });
  const user = db.prepare("SELECT id, username, email FROM users WHERE id =
  ?").get(req.user.userid);
  res.json({ user });
});

// /api/auth/logout - invalidation côté client
app.post('/api/auth/logout', (req, res) => {
  res.clearCookie("ourSimpleApp", {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: "strict"
  });
  res.status(200).json({ message: "Déconnexion réussie" });
});
```

Commentaire :

- **httpOnly** → illisible par JS (mitige XSS)
- SameSite=Strict → limite CSRF.
- exp → durée de vie courte (24h)

A.4 Service de données des challenges (challengeService.js)

```
// challenges/challengeService.js
const path = require('path');
const Database = require('better-sqlite3');
const db = new Database(path.join(__dirname, 'ctf.db'));

// DDL (au cas où la table n'existe pas)
db.prepare(`
  CREATE TABLE IF NOT EXISTS challenges (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    category TEXT NOT NULL,
    description TEXT NOT NULL,
    flag TEXT NOT NULL,
    points INTEGER NOT NULL
  )
`).run();

// Projection minimale (flag exclu)
function getAllChallenges() {
  return db.prepare('SELECT id, title, category, description,
points FROM challenges').all();
}

function getChallengeById(id) {
  return db.prepare('SELECT id, title, category, description,
points FROM challenges WHERE id = ?').get(id);
}

function addChallenge({ title, category, description, flag,
points }) {
  const stmt = db.prepare('INSERT INTO challenges (title,
category, description, flag, points) VALUES (?, ?, ?, ?,
?)');
  const result = stmt.run(title, category, description, flag,
points);
  return { id: result.lastInsertRowid, title, category,
description, points };
}

function verifyFlag(challengeId, submittedFlag) {
  const row = db.prepare('SELECT flag FROM challenges WHERE
id = ?').get(challengeId);
  return row ? row.flag === submittedFlag : false;
}

module.exports = { getAllChallenges, getChallengeById,
addChallenge, verifyFlag };
```

Commentaire :

- Le **flag** n'est jamais exposé via les listes/détails.
- verifyFlag compare côté serveur → sécurité.

A.5 Routes API challenges (Express) + contrôles d'entrée

```
// server.js - intégration des routes challenges
const challengeService =
require('./challenges/challengeService');

// Lister tous les challenges (public)
app.get('/api/challenges', async (req, res) => {
  try {
    const challenges = await
challengeService.getAllChallenges();
    res.json(challenges);
  } catch {
    res.status(500).json({ error: 'Erreur lors de la
récupération des challenges' });
  }
});

// Ajouter un challenge (à protéger si besoin)
app.post('/api/challenges', async (req, res) => {
  const { title, category, description, flag, points } =
req.body;
  if (!title || !category || !description || !flag ||
!points) {
    return res.status(400).json({ error: "Tous les champs
sont requis" });
  }
  try {
    const newChallenge = await
challengeService.addChallenge({ title, category, description,
flag, points });
    res.status(201).json(newChallenge);
  } catch {
    res.status(500).json({ error: 'Erreur lors de l'ajout du
challenge' });
  }
});

// Vérifier un flag
app.post('/api/challenges/:id/verify', async (req, res) => {
  const { submittedFlag } = req.body;
  try {
    const isValid = await
challengeService.verifyFlag(req.params.id, submittedFlag);
    res.json({ valid: isValid });
  } catch {
    res.status(500).json({ error: "Erreur lors de la
vérification du flag" });
  }
});
```

Commentaire :

- Projection minimale côté service, contrôle d'entrée côté route (400 si invalide)
- Penses à protéger POST (ajout) par auth/rôle si tu ajoutes une interface d'admin

A.6 Initialisation appli / sécurité / statique

```
// server.js - initialisation Express & SQLite
require('dotenv').config();
const express = require('express');
const jwt = require('jsonwebtoken');
const cookieParser = require('cookie-parser');
const bcrypt = require('bcrypt');
const cors = require('cors');
const path = require('path');
const db = require("better-sqlite3")("ourApp.db");
db.pragma("journal_mode = WAL"); // optimisation I/O

const app = express();
app.use(cors());
app.use(express.json());
app.use(cookieParser());

// Exposer éventuellement un frontend statique
app.use(express.static(path.join(__dirname,
'../frontend/public')));

// Exemple de route statique « protégée » par présence de JWT
(redirige sinon)
app.get('/acad', (req, res) => {
  if (req.user) return res.redirect('/dashboard.html');
  res.sendFile(path.join(__dirname,
'../frontend/public/acad.html'));
});

// 404 catch-all
app.use((req, res) => res.status(404).send("Page non
trouvée"));

// Lancement
app.listen(5000, () => console.log("Serveur démarré sur le
port 5000"));
```

Commentaire :

- **WAL** meilleurs perfs lecture/écriture concurrentes.
- Statique servi depuis frontend/public (si présent).

A.7 Schémas (ASCII)

A.7.1 Modèle de données

+-----+ users +-----+		+-----+ challenges +-----+
id INTEGER PK AUTOINC	1..*	id INTEGER PK AUTOINC
username TEXT UNIQUE	<-----	title TEXT NOT NULL
email TEXT UNIQUE		category TEXT NOT NULL
password TEXT NOT NULL		description TEXT NOT NULL
+-----+		flag TEXT NOT NULL
		points INTEGER NOT NULL
		+-----+

A.7.2 Séquence d'authentification

```
Client Serveur (Express) SQLite
| POST /api/auth/login (email,pwd) --->|
| SELECT * FROM users WHERE email=? -> verf bcrypt
| <--- OK/KO
| Set-Cookie: ourSimpleApp=JWT(HttpOnly,SameSite=Strict,Secure*)
| (*en prod)
| GET /api/auth/me ---> middleware lit cookie -> jwt.verify()
| <--- 200 { user } / 401 si invalide/expiré
|
| POST /api/auth/logout ---> clearCookie(ourSimpleApp)
| <--- 200
```


A.7 Schémas (ASCII)

A.7.1 Flux vérification de flag

```
Client API /verify challengeService SQLite
| POST /api/challenges/:id/verify ---> |
| body: { submittedFlag } |
| verifyFlag(id, submittedFlag) |
| SELECT flag WHERE id=? ->|--> row.flag
| compare local (==) |
| <--- { valid: true/false } |
```

A.8 Exemples de requêtes (cURL / Postman)

A.8.1 Inscription

```
curl -X POST http://localhost:5000/api/auth/register \
  -H "Content-Type: application/json" \
  -d
'{"username":"momo","email":"momo@example.com","password":"supermotdepasse"}'
```

A.8.2 Connexion (reçoit un cookie JWT)

```
curl -i -X POST http://localhost:5000/api/auth/login \
  -H "Content-Type: application/json" \
  -d
'{"email":"momo@example.com","password":"supermotdepasse"}'
# -> Note: le cookie ourSimpleApp est renvoyé par Set-Cookie
```

A.8.3 Profil authentifié

```
curl -X GET http://localhost:5000/api/auth/me \
  -H "Cookie: ourSimpleApp=<TOKEN_JWT_RECUPERER_LOGIN>"
```

A.8.4 Liste challenges (flag non exposé)

```
curl -X GET http://localhost:5000/api/challenges
```


A.8 Exemples de requêtes (cURL / Postman)

A.8.5 Vérifier un flag

A terminal window with a dark background and light green text. The command is: curl -X POST http://localhost:5000/api/challenges/42/verify \ -H "Content-Type: application/json" \ -d '{"submittedFlag":"CTF{abc123}"}'

```
curl -X POST http://localhost:5000/api/challenges/42/verify \
-H "Content-Type: application/json" \
-d '{"submittedFlag":"CTF{abc123}"}'
```

A.9 Optimisations (DB & API)

```
-- Index (optionnel) si filtrage par catégorie fréquent
CREATE INDEX IF NOT EXISTS idx_challenges_category ON
challenges(category);
```

```
// Projections minimales = moins d'I/O + moins de surface
d'attaque
db.prepare('SELECT id, title, category, description, points
FROM challenges').all();

// Requêtes préparées (better-sqlite3) = sûreté + perf
db.prepare('SELECT * FROM users WHERE email = ?').get(email);

// WAL (déjà activé dans server.js)
db.pragma("journal_mode = WAL");
```

Checklist qualité :

- Ne jamais renvoyer **flag** côté **API** publique.
- Valider toutes les entrées (types, longueur, listes de valeurs).
- Utiliser Secure : true en prod (HTTPS)
- Demander **JWT minimaux (userid, username, exp)**

A.10 Routes d'erreurs & 404

A code editor window with a dark background and light-colored text. It features three window control buttons (minimize, maximize, close) in the top-left corner. The code is written in a syntax-highlighted style with blue for comments, green for function names, and black for other identifiers and punctuation.

```
// 404 - catch-all
app.use((req, res) => {
  res.status(404).send("Page non trouvée");
});
```